



PÓS FIAP · ARQUITETURA DE SISTEMAS

Arquitetura de Software com Microserviços

Comunicação assíncrona · Filas · Serverless · Kubernetes

Estudo de caso: **Fiap Cloud Games** — um sistema real, em produção, que vamos abrir para ver cada conceito funcionando.

Prof. Vinicius Breda

01

PARTE 1

Os fundamentos

Antes de qualquer código ou ferramenta, precisamos de três ideias: **como dois sistemas conversam, o que é uma fila e por que alguém dividiria uma aplicação em várias.**

Sem isso, microsserviço vira palavra bonita. Com isso, vira decisão consciente.



Comunicação síncrona: eu falo e espero

Comunicação **síncrona** é aquela em que quem pede **fica parado esperando a resposta**. Enquanto o outro lado não responde, o solicitante não faz mais nada: ele está bloqueado.

É o modelo mais natural e o que todo programador aprende primeiro. Quando seu código chama um método, ele espera o retorno. Quando um sistema faz uma chamada HTTP para outro, a mesma coisa: a linha seguinte só executa quando a resposta chegar — ou quando estourar o tempo limite.

No dia a dia: é o telefonema. Você liga, a outra pessoa precisa estar disponível naquele exato momento, e você fica com o telefone no ouvido até ela responder. Se ela não atende, você não consegue o que queria. Se ela demora, você espera.

Quando é a escolha certa

Quando você **precisa da resposta para continuar**: consultar saldo antes de autorizar um saque, validar um login, buscar o preço de um produto para exibir na tela.

O problema que ela cria

Seu sistema fica **refém** do outro. Se ele cair, você cai junto. Se ele ficar lento, você fica lento. A disponibilidade de um vira a disponibilidade dos dois.



Comunicação assíncrona: eu aviso e sigo em frente

Na comunicação **assíncrona**, quem pede **não espera**. Ele deixa o recado em algum lugar confiável e volta imediatamente a trabalhar. Em algum momento — daqui a um milissegundo ou daqui a dez minutos — alguém pega aquele recado e executa a tarefa.

A mudança parece pequena, mas ela inverte uma relação importante: os dois sistemas **não precisam mais estar no ar ao mesmo tempo**. Isso se chama **desacoplamento temporal**, e é a propriedade que torna tudo o que veremos adiante possível.

No dia a dia: é a mensagem de WhatsApp. Você manda e guarda o celular. A pessoa pode estar dormindo, sem bateria ou sem sinal — a mensagem fica lá esperando. Quando ela abrir o aplicativo, vai receber. Você não ficou parado olhando a tela.

Quando é a escolha certa

Quando a tarefa **pode acontecer depois** sem prejudicar o usuário: enviar e-mail, gerar relatório, processar pagamento, atualizar um estoque, emitir nota fiscal.

O preço que se paga

A resposta deixa de ser o resultado final. Ela vira um **"recebi seu pedido"**. O usuário descobre o desfecho depois — por tela de acompanhamento, notificação ou e-mail.



Lado a lado: o que muda na prática

O mesmo cenário — uma compra que precisa ser paga — nos dois modelos:

	SÍNCRONO (CHAMADA DIRETA)	ASSÍNCRONO (FILA)
Quem espera	O solicitante fica bloqueado até a resposta.	Ninguém espera. O solicitante responde na hora.
Se o outro cai	A operação falha. O usuário vê erro.	O recado espera guardado. Processa quando voltar.
Se o outro fica lento	Você fica lento junto. O gargalo é contagioso.	Não te afeta. A fila só acumula um pouco mais.
Resposta ao usuário	"Compra aprovada." — resultado final.	"Compra recebida." — o desfecho vem depois.
Complexidade	Baixa. Fácil de escrever, testar e depurar.	Maior. Exige infraestrutura e novos cuidados.

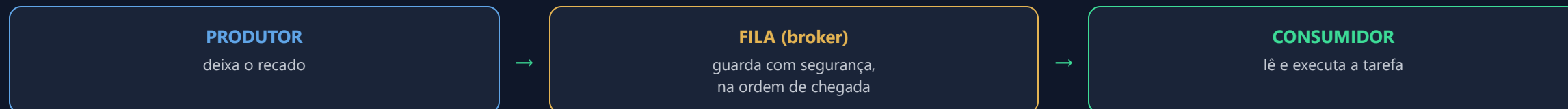
Não existe "o melhor". Existe o adequado para cada necessidade. Um sistema real usa os dois: síncrono onde precisa de resposta imediata, assíncrono onde a tarefa pode seguir seu curso.



O que é uma fila de mensagens

Uma fila é simplesmente **um lugar seguro onde recados ficam guardados até alguém poder atendê-los**. Quem escreve o recado chama-se **produtor**. Quem lê e executa chama-se **consumidor**. Entre os dois fica um programa especializado em guardar mensagens: o **message broker** — no nosso caso, o RabbitMQ.

A regra fundamental é esta: **o produtor nunca fala com o consumidor**. Ele entrega a mensagem ao broker e vai embora. O consumidor, por sua vez, não sabe quem produziu — ele só vê mensagens chegando. Nenhum dos dois conhece o endereço, a versão ou o horário de funcionamento do outro.



No dia a dia: é a senha do banco. Você chega, tira a senha e senta. Não precisa saber qual caixa vai te atender, nem se ele está no cafezinho. Se chegarem 50 pessoas de uma vez, a fila cresce — ninguém é mandado embora. O banco abre mais guichês para dar conta.



A fila não perde: o que ela garante

Guardar a mensagem é só metade do trabalho. A outra metade é **garantir que ela seja processada** mesmo quando algo dá errado.

1 · Persistência

A mensagem é gravada em disco. Se o broker reiniciar, ela continua lá.

Nada se perde porque "o servidor caiu".

2 · Confirmação (ack)

O consumidor só confirma depois de **terminar** o trabalho.

Sem confirmação, o broker considera que a tarefa não foi feita.

3 · Reentrega

Sem confirmação, a fila **entrega de novo** — para o mesmo consumidor ou para outro.

O trabalho não evapora silenciosamente.

No dia a dia: é o carteiro com uma carta registrada. Ele só dá baixa quando alguém assina o recebimento. Se ninguém atende, ele volta amanhã. E tenta de novo. A entrega não é dada como feita só porque ele passou na porta.

Guarde esta frase: a fila promete entregar pelo menos uma vez — nunca exatamente uma vez. Essa diferença, que parece detalhe, é o assunto do próximo slide.



A mesma mensagem pode chegar duas vezes

Se a fila reentrega quando não recebe confirmação, existe um cenário inevitável: o consumidor **fez o trabalho**, mas a confirmação se perdeu no caminho. Para o broker, aquela tarefa não foi concluída — então ele entrega a mensagem outra vez.

Num envio de e-mail, o estrago é pequeno: o cliente recebe dois e-mails. Num **pagamento**, o estrago é sério: o cliente é cobrado duas vezes. Não dá para "desligar" essa possibilidade — ela é consequência direta de como filas funcionam em qualquer tecnologia.

A solução não está na fila. Está no consumidor. Ele precisa ser **idempotente**: capaz de receber a mesma mensagem várias vezes **produzindo sempre o mesmo resultado**, como se tivesse recebido uma só.

No dia a dia: o boleto tem um número único. Se o carteiro entregar a mesma conta duas vezes, você olha o número, percebe que já pagou aquele boleto e não paga de novo. Você é idempotente — não por sorte, mas porque existe um identificador que permite reconhecer o repetido.

Na prática: todo consumidor precisa de uma **chave de negócio** (número do pedido, id da compra) e de uma pergunta antes de agir: "eu já processei isso?". Sem essa chave, não há idempotência possível.



E quando a mensagem nunca funciona?

Reentregar resolve falhas **passageiras**. Mas e quando o erro é permanente — um dado corrompido, uma regra que nunca vai passar? Tentar para sempre travaria a fila inteira.



Por que esperar entre as tentativas

Se o banco de dados caiu, tentar de novo no mesmo milissegundo falha igual. O intervalo crescente — 1s, 5s, 10s — dá tempo para o problema se resolver sozinho.

Fila de erro (dead-letter queue)

Esgotadas as tentativas, a mensagem **não é descartada**: vai para uma fila separada. Alguém investiga, corrige a causa e reprocessa depois. O problema fica visível em vez de sumir.

No dia a dia: o carteiro tenta três dias seguidos. Na quarta, leva a carta de volta para a agência e registra "destinatário ausente". A carta não foi jogada fora — está guardada, catalogada, esperando alguém resolver.



O monólito: uma aplicação, um deploy

No monólito, **todo o sistema é uma aplicação só**. Cadastro de usuários, catálogo de jogos, pagamento e relatórios vivem no mesmo projeto, compilam juntos, sobem juntos e usam o mesmo banco de dados.

É assim que praticamente todo software começa — e está **certo que comece assim**. Chamar um método é mais simples que publicar um evento; uma transação de banco é mais simples que coordenar dois serviços; depurar um processo é mais simples que seguir um rastro por cinco máquinas.

Onde o monólito é excelente

Time pequeno. Cinco pessoas no mesmo repositório não atrapalham umas às outras.

Produto novo. Quando o negócio ainda está mudando toda semana, fronteiras rígidas atrapalham.

Transações simples. Um BEGIN/COMMIT resolve a consistência de tudo.

Onde ele começa a doer

Deploy. Mudar uma linha do relatório exige subir o sistema inteiro — inclusive o pagamento.

Escala. Se só o pagamento está sobrecarregado, você precisa duplicar tudo para atendê-lo.

Falha. Um vazamento de memória no relatório derruba o checkout junto.

Time grande. 60 pessoas no mesmo código viram fila de merge e medo de mexer.



Microsserviços: vários sistemas pequenos e autônomos

Microsserviços são a decisão de **quebrar a aplicação em serviços independentes**, cada um responsável por uma parte do negócio, com **seu próprio banco de dados** e seu próprio ciclo de vida.

A palavra que mais importa aqui não é "micro" — é **autônomo**. Um serviço só é microsserviço de verdade se puder ser alterado, testado, implantado e escalado **sem combinar nada com os outros**. Se para subir o serviço de pagamento você precisa avisar o time de catálogo, você não tem microsserviços: tem um monólito espalhado em vários servidores.

Banco próprio

Ninguém lê a tabela do vizinho. Precisa de um dado? Pede pelo contrato, nunca pelo banco.

Deploy independente

Cada serviço sobe no seu ritmo, sem janela coordenada e sem travar os outros times.

Escala seletiva

Só o serviço sob pressão ganha mais cópias. Os demais seguem com o que têm.

No dia a dia: *um shopping. A loja de roupas, a praça de alimentação e o cinema são negócios separados — cada um com seu caixa, seu estoque e seu horário. Se o cinema fecha para reforma, a praça de alimentação continua vendendo. No monólito, seria uma loja gigante onde a reforma de um setor fecha a porta de todos.*



Monólito x Microserviços, ponto a ponto

CRITÉRIO	MONÓLITO	MICROSSERVIÇOS
Deploy	Tudo junto. Uma linha alterada faz o sistema inteiro subir.	Cada serviço no seu ritmo, sem combinar com ninguém.
Escala	Duplica tudo, mesmo que só um módulo precise.	Multiplica só o serviço sob carga.
Falha	Um erro grave pode derrubar o sistema todo.	Falha isolada. O resto continua de pé.
Banco	Um banco compartilhado; todos acoplados no schema.	Um banco por serviço; schema privado.
Times	Todos no mesmo código, disputando espaço.	Times autônomos, donos do serviço ponta a ponta.
Depuração	Simples. Um log, um stack trace.	Difícil. O rastro atravessa vários serviços.
Consistência	Imediata. Uma transação resolve.	Eventual. O dado se ajusta em instantes, não no mesmo momento.
Infra	Um servidor e um banco bastam.	Broker, orquestrador, monitoramento, pipelines.



Afinal, quando usar cada um?

Esta é a pergunta que realmente importa — e a resposta honesta contraria a moda.

✓ Fique no monólito quando

- O time cabe em uma sala e todos conhecem o sistema inteiro.
- O produto é novo e as regras ainda mudam toda semana.
- O volume é previsível e um servidor dá conta com folga.
- Você ainda não entende bem as fronteiras do negócio.
- Não há equipe para operar broker, cluster e observabilidade.

✓ Parta para microsserviços quando

- Vários times precisam entregar sem esperar uns aos outros.
- Partes do sistema têm **escalas muito diferentes** — o pagamento explode na Black Friday, o cadastro não.
- Uma área precisa de disponibilidade muito maior que o resto.
- As fronteiras do negócio já estão claras e estáveis.
- Existe maturidade de infraestrutura para sustentar a operação.

O erro mais caro da carreira de muita gente: começar um produto novo já com doze microsserviços. Sem conhecer as fronteiras do negócio, elas saem erradas — e mover uma fronteira entre serviços é dezenas de vezes mais caro do que mover uma pasta dentro de um monólito.

Em uma frase: um monólito bem organizado é melhor que microsserviços mal divididos. Microsserviço é a resposta para um problema de escala e organização — não um troféu de modernidade.

02

PARTE 2 • ESTUDO DE CASO

Fiap Cloud Games

Agora vamos abrir um sistema **real**, em produção, e encontrar cada conceito da Parte 1 funcionando: a comunicação assíncrona, a fila, a reentrega, a idempotência e a divisão em serviços autônomos.

Uma plataforma de venda de jogos digitais. O usuário compra um jogo, o pagamento é processado e ele recebe a confirmação por e-mail.



Os serviços do Fiap Cloud Games

Quatro peças independentes. Cada uma faz **uma coisa** e tem o seu próprio dado.

API Jogos

Catálogo, estoque e compra. É por aqui que a jornada começa.

banco próprio

API Usuários

Cadastro e autenticação. Emite o token que protege as demais.

banco próprio

API Pagamentos

Processa o pagamento da compra e informa o desfecho.

banco próprio

Azure Function

Envia o e-mail de confirmação. Sem servidor, sem banco.

serverless

No centro: o RabbitMQ

Todos falam com a fila; **ninguém fala com o vizinho**. É isso que permite derrubar, atualizar ou multiplicar qualquer serviço sem avisar os outros.

O teste do microsserviço

Se a API de Jogos precisasse chamar a de Pagamentos *diretamente* para funcionar, as duas estariam amarradas — e seria aquele "monólito distribuído" do slide 11.



As filas do projeto, no RabbitMQ de verdade

Quatro filas. O nome segue um padrão: **quem-consome** . **o-que-aconteceu**.

 RabbitMQ™
RabbitMQ 3.13.7 Erlang 26.2.5.16

Refreshed 2026-10-04 22:09:17 Refresh every 5 seconds Virtual host All Cluster rabbit@d49f4f706817 User fcg Log out

Overview Connections Channels Exchanges Queues and Streams Admin

Queues

▼ All queues (4)

Pagination

Page 1 of 1 - Filter: ☐ Regexp ☐ Displaying 4 items , page size up to: 100

Painel do RabbitMQ · as 4 filas em execução

FILA	QUEM CONSUME	PARA QUÊ
pagamentos.pagamento-solicitado	Pagamentos	Recebe o pedido de pagamento vindo da compra
jogos.pagamento-aprovado	Jogos	Baixa o estoque e conclui a compra
notificacoes.pagamento-aprovado	Pagamentos	Dispara o e-mail de confirmação
jogos.pagamento-recusado	Jogos	Marca a compra como recusada



O que viaja pela fila: os eventos

Pela fila não trafega "código" — trafega um **aviso de que algo aconteceu**. Repare que todos os nomes estão no **passado**: eles descrevem fatos consumados, não ordens.

PagamentoSolicitado

"Uma compra foi registrada e precisa ser paga."

Leva: id da compra, jogo, usuário, quantidade e valor.

PagamentoAprovado

"O pagamento daquela compra foi aprovado."

Leva: id da compra e identificador do pagamento.

PagamentoRecusado

"O pagamento daquela compra não passou."

Leva: id da compra e o motivo da recusa.

Dois campos presentes em todos

O **id da compra**, que permite reconhecer mensagem repetida — é a chave de idempotência do slide 08.

E o **CorrelationId**, um código que nasce no clique do usuário e acompanha todos os eventos daquela jornada.

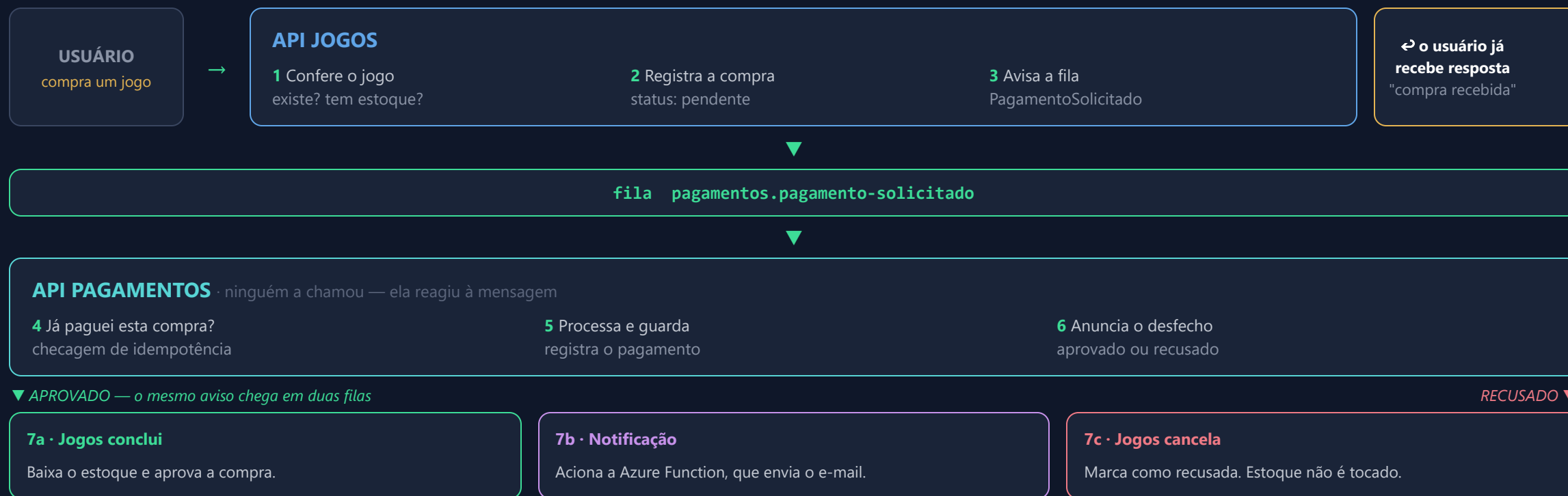
Por que isso importa

É só isso que os serviços compartilham. Nenhum vê o banco do outro, nenhum chama o método do outro. **O contrato é a mensagem** — e é o que mantém os times independentes.



A jornada completa de uma compra

Do clique do usuário ao e-mail de confirmação, atravessando **três serviços** sem que nenhum chame o outro diretamente.





O detalhe que mais confunde — e o mais importante

Quando o usuário clica em "comprar", a resposta volta no **passo 3** — antes de o pagamento sequer começar. E a compra fica, por alguns instantes, com o status **pendente**.

Quem está acostumado com sistemas síncronos acha que deu errado. Não deu. Essa é exatamente a troca que fizemos: o usuário recebe uma resposta **rápida e garantida** ("recebemos seu pedido") em vez de uma resposta **lenta e incerta** ("aguarde enquanto falamos com o banco...").

Se fosse síncrono

O usuário esperaria o pagamento inteiro na tela.

Pagamento fora do ar = **compra perdida** e erro na cara do cliente.

Pagamento lento = site lento para todo mundo.

Como realmente é

O usuário recebe confirmação em milissegundos.

Pagamento fora do ar = a mensagem **espera na fila**; quando o serviço volta, processa normalmente.

O cliente acompanha o desfecho pelo pedido ou pelo e-mail.

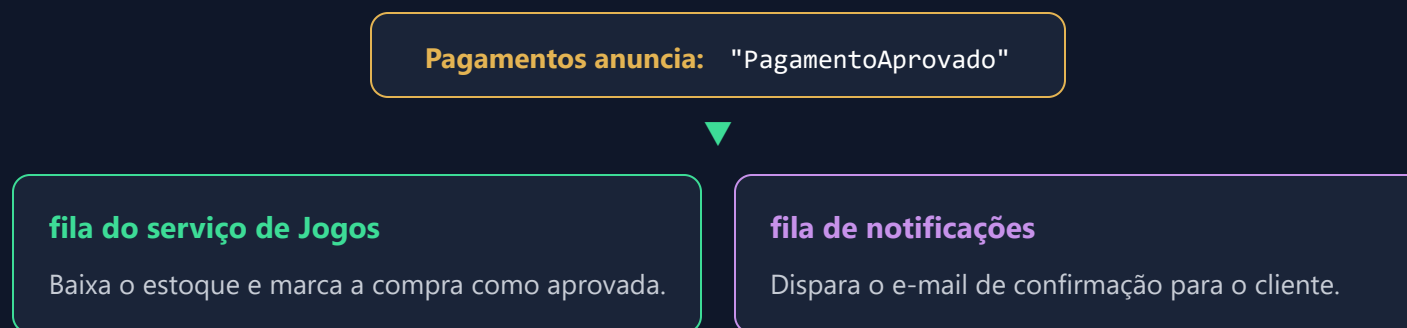
No dia a dia: é o iFood. Você finaliza o pedido e a tela confirma na hora — mas a comida ainda não foi preparada, nem saiu para entrega. Ninguém espera de olho na tela até o entregador chegar: você acompanha o status.



Um aviso, vários interessados

Quando o pagamento é aprovado, duas coisas precisam acontecer: **a compra deve ser concluída** e **o cliente deve ser avisado por e-mail**. Poderíamos mandar dois recados. Mandamos **um só**.

O serviço de Pagamentos simplesmente anuncia: "o pagamento foi aprovado". Quem tiver interesse nesse fato cria a sua própria fila e escuta. Hoje são dois interessados — e o serviço de Pagamentos **não sabe o nome de nenhum deles**.



A vantagem prática: quer um serviço de antifraude ou de relatórios ouvindo essa mesma informação amanhã? Basta ele criar a fila dele. **Nenhuma linha** muda no serviço de Pagamentos — e nenhum deploy dele é necessário.



Idempotência aplicada: não cobrar duas vezes

Aquele conceito do slide 08, agora resolvido em código real. Num sistema de pagamento, é a diferença entre um cliente satisfeito e um processo no Procon.

1 · A pergunta antes de agir

Ao receber o pedido, o serviço consulta: "**já existe pagamento para esta compra?**"

Se existe, devolve o que já foi feito e **não cobra de novo**.

2 · A trava no banco

A coluna do id da compra tem **índice único**. Mesmo que duas cópias do serviço tentem gravar ao mesmo tempo, o banco só aceita uma.

3 · Do outro lado também

O serviço de Jogos só conclui compras que estejam **pendentes**. Aviso repetido é ignorado — senão o estoque baixaria duas vezes.

Por que a trava no banco, se já perguntamos antes? Porque entre a pergunta e a gravação existe um intervalo. Com várias cópias do serviço rodando em paralelo, duas podem perguntar ao mesmo tempo, receber "não existe" e tentar gravar. O índice único é a última linha de defesa — e é ela que torna seguro multiplicar o serviço.

Guarde: idempotência não é um detalhe de implementação. É o que torna possível **escalar** e **reentregar** com segurança — os dois pilares de tudo que vimos até aqui.



Garantindo que o aviso nunca se perca

Existe uma falha silenciosa que todo sistema com fila enfrenta. Registrar a compra no banco e avisar a fila são **duas ações separadas**. E se a primeira funcionar e a segunda falhar?

A compra fica gravada, mas **ninguém nunca será avisado** de que ela precisa ser paga. Ela permanece pendente para sempre. O cliente pagou com a atenção dele e não recebe nada — e o pior: **não há erro em lugar nenhum**.

✗ Sem proteção

1. Grava a compra no banco ✓
2. **O serviço cai neste instante** ⚡
3. O aviso nunca é publicado ✗

Resultado: compra pendente eternamente.

✓ Com a caixa de saída

1. Grava a compra **e o aviso juntos**, na mesma operação ✓
2. Confirmada a gravação, um processo em segundo plano entrega o aviso à fila ✓

Ou os dois existem, ou nenhum existe.

No dia a dia: é a caixa de saída do seu e-mail. Você escreve sem internet e clica em enviar — a mensagem fica guardada na caixa de saída. Quando a conexão volta, ela parte sozinha. Você não precisou lembrar de reenviar, e a mensagem não se perdeu.

03

PARTE 3

Serverless

Nem toda tarefa merece um serviço inteiro de pé 24 horas por dia. Para o trabalho **curto, esporádico e disparado por um evento**, existe um modelo mais barato e mais simples.

No Fiap Cloud Games, esse trabalho é o envio do e-mail de confirmação.



SERVERLESS

O que é serverless de verdade

O nome engana: **o servidor existe**. O que muda é que você **não gerencia nenhum**. Você entrega apenas uma função — um pedaço de código com uma entrada e uma saída — e a nuvem cuida de todo o resto.

A função não fica rodando o tempo todo. Ela **dorme** até um gatilho acordá-la: uma chamada HTTP, um arquivo que chegou, um horário agendado, uma mensagem na fila. Terminou a execução, ela some.

Você entrega

Só a função. Nada de servidor, sistema operacional ou configuração de escala.

A nuvem entrega

Infraestrutura, atualizações de segurança, balanceamento e escala automática.

Você paga

Por **execução**. Ninguém usou hoje? O custo é **zero** — não existe máquina ociosa sendo cobrada.

O trade-off: cold start

Depois de um tempo parada, a primeira execução demora mais, porque a nuvem precisa "ligar" o ambiente. Para enviar um e-mail, irrelevante. Para responder a um clique do usuário, pode incomodar.

Quando NÃO usar

Processos longos, que guardam estado na memória ou que recebem carga alta e constante. Nesses casos, um serviço tradicional sai mais barato e mais previsível.



Como as empresas usam serverless

O padrão se repete em todos os casos: **tarefa curta, disparada por um evento, com volume irregular.**

Notificações

E-mail, SMS e push disparados por acontecimentos do negócio.

É o nosso caso no FCG.

Processamento de mídia

Usuário sobe uma foto e uma função gera as miniaturas e comprime. Plataformas de vídeo usam esse modelo em larga escala.

Integrações e webhooks

Receber o retorno de um gateway de pagamento ou de um sistema parceiro — tráfego imprevisível, execução de segundos.

Rotinas agendadas

Relatório diário, limpeza de base, expiração de carrinho. Substitui o servidor que existia só para rodar uma rotina de madrugada.

Processamento de dados

Transformar e mover informação entre sistemas conforme ela chega, sem manter um cluster ligado o dia inteiro.

Picos extremos

Black Friday, estreia de jogo, venda de ingresso. Escala para milhares de execuções e volta a zero sozinha.

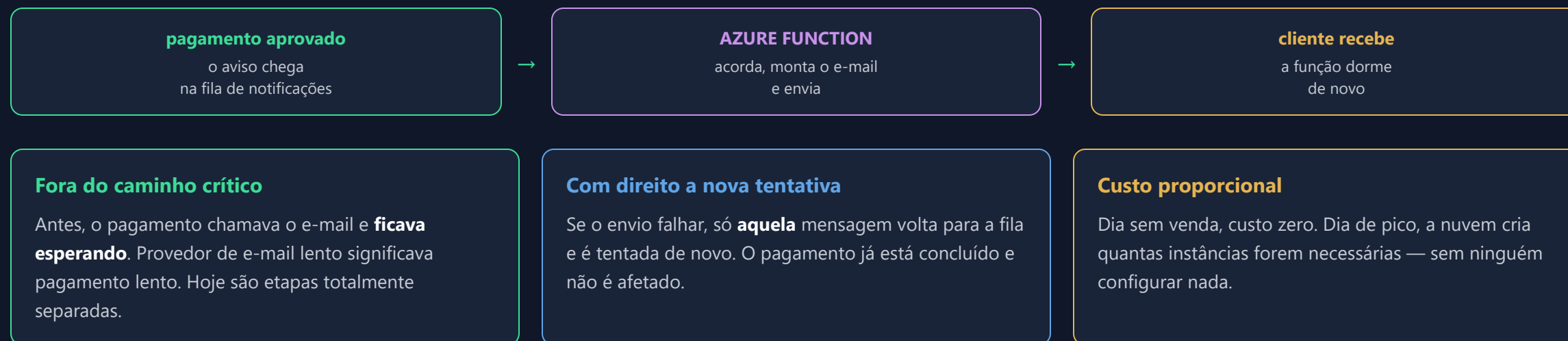
Os principais fornecedores: *Azure Functions (Microsoft) · AWS Lambda (Amazon) · Cloud Functions (Google).* O conceito é o mesmo nos três — muda o nome e os gatilhos disponíveis.



SERVERLESS

A Azure Function no Fiap Cloud Games

No nosso sistema, enviar o e-mail de confirmação é uma tarefa **curta, ocasional e que não pode travar um pagamento**. Manter um serviço inteiro de pé só para isso seria desperdício — é o caso de uso perfeito para serverless.



04

PARTE 4

Kubernetes e escala

Já temos serviços independentes conversando por filas. Falta responder: **quem coloca cada um no ar, mantém de pé e multiplica quando a demanda aumenta?**



KUBERNETES

O que é Kubernetes

Kubernetes é o **orquestrador**: o programa que coloca seus serviços para rodar, **vigia se continuam vivos**, recria o que cair e cria cópias extras quando a carga aumenta.

Você não diz "ligue este servidor". Você **declara o que deseja** — "quero três cópias da API de Jogos sempre no ar" — e ele se encarrega de manter essa realidade, inclusive às três da manhã, sem acordar ninguém.

Cluster

O conjunto de máquinas disponíveis para rodar suas aplicações.

Pod

A menor unidade: uma cópia do seu serviço em execução.

Deployment

Onde você declara quantas cópias quer manter no ar.

Service

Um endereço fixo que distribui as chamadas entre as cópias.

Health check

Pergunta periódica: "você está bem?". Quem não responde é substituído.

Autoescala (HPA)

Cria e remove cópias sozinho, conforme o consumo sobe ou desce.

Relação com serverless: os dois escalam sozinhos. A diferença é que o Kubernetes mantém seu serviço **sempre de pé** — ideal para quem recebe tráfego constante; serverless só existe durante a execução — ideal para a tarefa esporádica.



KUBERNETES

Cada microsserviço com o seu próprio gerenciamento

No FCG, cada serviço tem **sua própria configuração de cópias e de autoescala** — independentes, inclusive na hora de crescer.

Nome	Namespace	Pronto	Idade	CPU	Memória
pagamentos-api	fiap8s	3/3	2 horas	0%	17%
usuarios-api	fiap8s	2/2	Uma hora	3%	12%
jogos-api	fiap8s	3/3	50 minutos	0%	11%

Cluster do Fiap Cloud Games · autoescala por serviço

Independência real

A API de Jogos pode estar com 5 cópias enquanto a de Usuários roda com 1. Cada uma cresce conforme a **sua** demanda.

Autocorreção

Cópia que parar de responder ao health check é removida e recriada automaticamente, sem intervenção humana.

Entrega automatizada

O pipeline compila, empacota e publica a nova versão no cluster — sem ninguém acessar servidor manualmente.



Onde filas e escala se encontram

Aqui as duas metades da aula se juntam. Imagine **mil compras chegando de uma vez** numa promoção. As mensagens se acumulam na fila e o autoescala percebe que o serviço de Pagamentos está sob pressão.

Ele cria mais cópias. E aqui está a parte elegante: **a fila distribui sozinha**. Cada mensagem é entregue a **uma única** cópia, então três cópias processam três vezes mais rápido — **sem nenhuma configuração extra**.



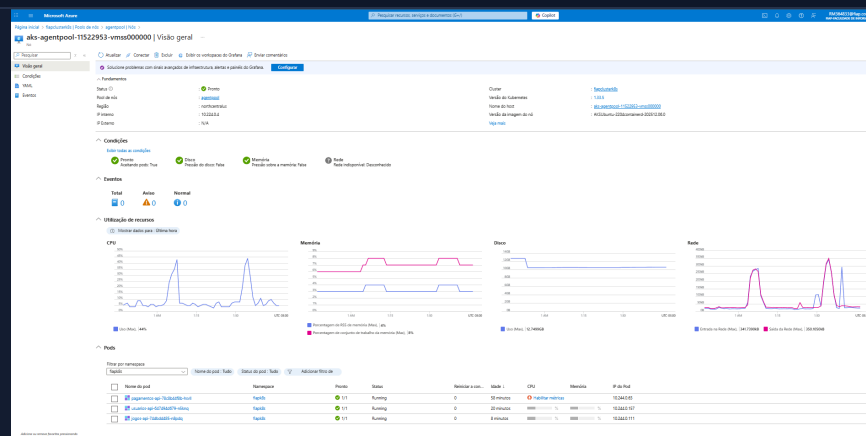
A fila funciona como amortecedor: num pico de dez vezes o normal, as mensagens esperam enquanto as cópias sobem. Ninguém recebe erro, nada se perde — só demora um pouco mais. Num sistema síncrono, esse mesmo pico viraria uma enxurrada de falhas na tela do cliente.

Só funciona por causa do slide 21: com três cópias processando em paralelo e reentregas acontecendo, a mesma mensagem ser processada duas vezes deixa de ser hipótese. **É a idempotência que torna a escala segura.**



Enxergando um sistema distribuído

Quando a jornada atravessa três serviços e uma função, **não existe um único log** contando a história inteira. Ela precisa ser remontada.



Telemetria do cluster em tempo real

O código que liga tudo

O **CorrelationId** nasce no clique e viaja em todos os eventos. Filtrar por ele reconstrói a jornada completa de uma compra.

O que monitorar nas filas

Tamanho da fila e **idade da mensagem mais antiga**. Fila crescendo significa consumidor sem capacidade — alerta antes do cliente reclamar.

A fila de erro é um termômetro

Mensagens acumulando ali indicam falha permanente. Em operação saudável, ela fica vazia.



O que essa arquitetura entrega

Cada benefício tem um **mecanismo concreto** por trás — nenhum deles é automático por "usar microsserviços".

Baixo acoplamento

Nenhum serviço conhece o outro. Todos falam com a fila.

Resiliência

Serviço fora do ar não derruba a operação: a fila guarda e entrega depois.

Nada se perde

Reentrega, fila de erro e caixa de saída garantem que nenhum pedido evapore.

Seguro para repetir

Idempotência impede cobrança e baixa de estoque em duplicidade.

Escala horizontal

Cada serviço cresce sozinho, e a fila distribui o trabalho entre as cópias.

Extensível

Novos interessados num evento entram sem que ninguém altere quem publica.

Para fechar com honestidade: tudo isso tem um preço — consistência eventual, mensagens duplicadas, depuração distribuída e muito mais infraestrutura para operar. Microsserviços e filas resolvem problemas **reais** de escala e organização. Se o seu sistema ainda não tem esses problemas, essa arquitetura vai cobrar o custo sem entregar o benefício.



RECAPITULANDO

Comunicação

Síncrono quando você precisa da resposta. Assíncrono quando a tarefa pode seguir seu curso.

Microserviços

Serviços autônomos, com banco próprio e deploy independente — quando o problema justifica.

Kubernetes

Mantém os serviços de pé e multiplica cópias conforme a demanda.

Filas

Guardam, reentregam e isolam falhas. Exigem consumidores idempotentes.

Serverless

Para a tarefa curta e esporádica, sem servidor para administrar.

E o principal

Escolher a arquitetura adequada ao problema — não a mais moderna.

Do clique à compra aprovada — sem ninguém esperando bloqueado.